

No puedo generar un archivo de Google Docs directamente desde aquí, pero te dejo todo el contenido formateado y estructurado a continuación. Solo tienes que copiarlo y pegarlo en un documento en blanco.



CHULETA DE REFERENCIA: EXAMEN LSEL (UPM)

1. CONTROL DE VERSIONES (GIT & GITHUB)

Requisito típico: Crear repositorio privado `examenLSEL202X-XX`, añadir a `pmalagon` como colaborador, clonar y subir código base.

Flujo de comandos básicos:

Bash

```
# 1. Clonar el repositorio vacío en tu ordenador
```

```
clone
```

```
# -> AHORA COPIA Y DESCOMPRIME EL CÓDIGO BASE DEL EXAMEN EN LA CARPETA <-
```

```
# 2. Añadir los archivos y forzar los .gitkeep (suele ser necesario en output)
```

```
# 3. Crear el commit y subir
```

```
"Commit inicial: volcado del código base"
```

Si te piden trabajar en una rama específica (ej. `test_snake`):

Bash

```
# Crear y cambiar a la rama
```

```
# ...hacer cambios...
```

```
# Subir la rama
```

2. TESTING UNITARIO Y DEPURACIÓN (CEEDLING)

Objetivo: Arreglar el código fuente (`src/`) para que pase los tests. Nunca modifiques los tests a menos que te lo pidan explícitamente.

Comandos de Ceedling en Terminal:

- Ejecutar todos los tests: `ceedling test:all`
- Ejecutar un test específico: `ceedling test:fsm_legacy` (ejemplo)
- Limpiar compilación (si hay errores raros): `ceedling clobber`
- Comprobar cobertura: `ceedling gcov:all utils:gcov`

Cómo leer los tests (CMock y Unity):

- **Mocks (Simulaciones):** Si el test dice `funcionX_ExpectAnyArgsAndReturn(1)`, tu código TIENE que llamar a `funcionX` y esta devolverá un 1 en esa prueba. Si dice `_Ignore()`, no importa cuántas veces se llame.
- **Asserts (Comprobaciones):** * `TEST_ASSERT_EQUAL(esperado, actual)`
 - `TEST_ASSERT_TRUE(condicion)`
 - `TEST_ASSERT_NOT_NULL(puntero)`

3. RASPBERRY PI: EJECUCIÓN Y DEBUGGING REMOTO

Credenciales por defecto: IP: 10.8.42.1XX (32-bits) o 10.8.42.2XX (64-bits). Usuario: pi, Contraseña: raspberry.

Paso a paso para depurar:

1. **Compilar para la Raspi:** En tu PC, ejecuta `make PLATFORM=rpi_hat`. Si pruebas un sample específico: `make PLATFORM=rpi_hat SAMPLE_FILE=sample_display`.
2. **Conexión SSH:** Abre un terminal y conéctate: `ssh pi@10.8.42.XXX`.
3. **Lanzar GDB Server (En la Raspi):**

```
Bash
```

6. (Nota: si usas NFS, la ruta será algo como `/mnt/nfs_export/snake`)

7. **Conectar VSCode (En tu PC):**

- Revisa `launch.json`. Asegúrate de que `miDebuggerServerAddress` tiene la IP correcta de la Raspi.
- Pon un *breakpoint* en tu código.
- Ve a la pestaña "Run and Debug" y lanza "Debug Linux Remote".
- **LLAMA AL PROFESOR PARA QUE FIRME.**

4. STM32F411E: IMPLEMENTACIÓN CON HAL

Esta parte evalúa código C y manejo de periféricos. Revisa las hojas de datos o esquemas si te los dan.

A. Comunicaciones I2C (Ej. Acelerómetro LSM303) ¡Cuidado! Las direcciones I2C de 7 bits **tienen que desplazarse 1 bit a la izquierda (`<< 1`)** cuando usas las funciones de la HAL.

- Ejemplo de dirección: `0x19` -> `(0x19 << 1)`.

C

```
// Transmitir (Escribir)
uint8_t          2

// [cite: 8221]

// Recibir (Leer un registro específico)
uint8_t          6
// 1. Apuntar al registro que queremos leer

// 2. Leer los datos

8248]

// Alternativa más directa (Mem_Read / Mem_Write):

// [cite: 8259, 8262]
```

B. Interrupciones GPIO (EXTI)

No leas el estado del botón en un bucle `while(1)`. Utiliza la rutina de servicio de interrupción (ISR) y *flags* volátiles.

```
C
volatile uint8_t          0

// Esta función la llama la HAL automáticamente cuando hay un flanco (interrupción)
void HAL_GPIO_EXTI_Callback uint16_t          // [cite: 9911]
if          // Si es el botón USER (PC13) [cite: 9904]
    1

// En la función de lógica de entrada (ej. update_new_heading):
if
    // Cambiar dirección
    0 // Limpiar flag
```

5. GITHUB ACTIONS (CI/CD)

Si piden automatización, los archivos van en la ruta `.github/workflows/archivo.yml`.

Ejemplo de YAML para correr tests al hacer Push:

```
YAML
name: Test and Build
on:
  push:
    cite_start branches: "main" # Se ejecuta al hacer push a main [cite: 11058]

jobs:
  test:
    cite_start runs-on: ubuntu-24.04 # [cite: 11146]
    steps:
      - name: Checkout del código
        cite_start uses: actions/checkout@v4 # [cite: 11167]

      - name: Instalar Ceedling
        run: |
          sudo apt-get update
          sudo apt-get install -y-qq ruby # [cite: 11188]
          sudo gem install ceedling -v 1.0.1 # [cite: 11189]

      - name: Ejecutar Tests
        run: ceedling test:all
```

Sintaxis del Badge (Chapa) para el README.md:

Tienes que sustituir `<user>`, `<repo>` y el nombre del archivo `.yml`.

```
Markdown
![[Nombre
Acción https://github.com/<user>/<repo>/actions/workflows/<archivo.yml>/badge.svg https://git
hub.com/<user>/<repo>/actions/workflows/<archivo.yml>
```

(Ejemplo de referencia)